Counting, Carrying, and Circuits

COMSC 2043 • Nov 10, 2026

1. Carrying is the same in every base. A base-b system has b symbols; the biggest is $b-1$. When a place runs out of symbols, write 0 there and add 1 to the next place.

	biggest digit +1	**two biggest +1**	**place values**
base 10	$9+1=10$	$99+1=100$	$1, 10, 100$
base 8	$7+1=10$	$77+1=100$	$1, 8, 64$
base 16	$F+1=10$	$FF+1=100$	$1, 16, 256$
base 2	$1+1=10$	$11+1=100$	$1, 2, 4, 8$

"10" always means *one group of b, none left over.* The marks could be anything: circle, star, face, fish.

2. Add one column at a time, right to left, then move sideways. Line the digits up. Write any carry above the next column.

place	100s	10s	1s
carry	1		
		5	2
+		5	3
	1	0	5

Ones: $2+3=5$.
Tens: $5+5=10$. Write 0, **carry 1**.
100s: nothing to add, so the carry drops down: 1.
Answer: 105.

Binary uses the identical steps, with a smaller table: $0+0=0,\ 0+1=1,\ 1+0=1,\ 1+1=$ `10` (write 0, carry 1).

place	16s	8s	4s	2s	1s
carry	1	1	1	0	
		1	0	1	1
+		0	1	1	0
	1	0	0	0	1

1s: $1+0=1$. **2s:** $1+1=10$, write 0, carry 1.
4s: $0+1+1=10$, write 0, carry 1.
8s: $1+0+1=10$, write 0, carry 1.
16s: the carry drops down: 1.
Answer: `10001` $(11+6=17)$.

Multiply the same way you do in decimal: one partial product per digit of the bottom number, each shifted left, then add the columns. A bit times a bit is $0\times0=0,\ 0\times1=0,\ 1\times0=0,\ 1\times1=1$.

	1	0	1	
×		1	1	
	1	0	1	101×1
1	0	1	0	101×1, shifted
1	1	1	1	$5\times3=15$

Try it (check in decimal): `0111 + 0101` `1111 + 0001` `110 × 101`

---

3. Boolean algebra. Values 0 and 1. **AND** $A\cdot B$ is 1 only if both are 1. **OR** $A+B$ is 1 if at least one is 1. **NOT** $\overline{A}$ flips. **XOR** $A\oplus B$ is 1 when the inputs differ.

Careful: in Boolean algebra $1+1=1$ (OR). In binary arithmetic $1+1=10$ (add). Always say which system you are in.

4. Half adder (adds two bits). Sum $S = A \oplus B$. Carry $C = A \cdot B$. Read CS as a two-bit number: it equals $A + B$.

A	B	C	S	$A+B$ in binary
0	0	0	0	`00`
0	1	0	1	`01`
1	0	0	1	`01`
1	1	1	0	`10`

5. Full adder (adds A, B, and a carry-in). $S = A \oplus B \oplus C_{in}$ $\quad C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$.

A	B	C_{in}	C_{out}	S	$A+B+C_{in}$
0	0	0	0	0	`00`
0	0	1	0	1	`01`
0	1	0	0	1	`01`
0	1	1	1	0	`10`
1	0	0	0	1	`01`
1	0	1	1	0	`10`
1	1	0	1	0	`10`
1	1	1	1	1	`11`

All eight input rows are listed, so the table is a **proof**, not just an example.

6. Chaining adders. A full adder per bit, each carry feeding the next: a **ripple-carry adder**. The carry must travel through every stage, so about $2n$ gate delays for n bits (8 bits: 16; 16 bits: 32). **Carry lookahead** computes the carries directly from *generate* $G_i = A_i B_i$ and *propagate* $P_i = A_i \oplus B_i$:

$$C_1 = G_0 + P_0 C_0, \qquad C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0.$$

A 16-bit lookahead adder (four 4-bit blocks plus a lookahead unit) finishes in about 8 gate delays instead of 32, at the cost of more hardware.

Check yourself (write your answers on the back or in your notes):

1. In your own words, what does "carry" mean?
2. Which gate makes the sum bit of a half adder? Which makes the carry?
3. Fill in the full-adder row $A = 1,\ B = 1,\ C_{in} = 1$ without looking. Why is the answer `11`?
4. Why is one correct example not enough to prove a circuit works? What would be?
5. Our delay model leaves something out. Name one thing.